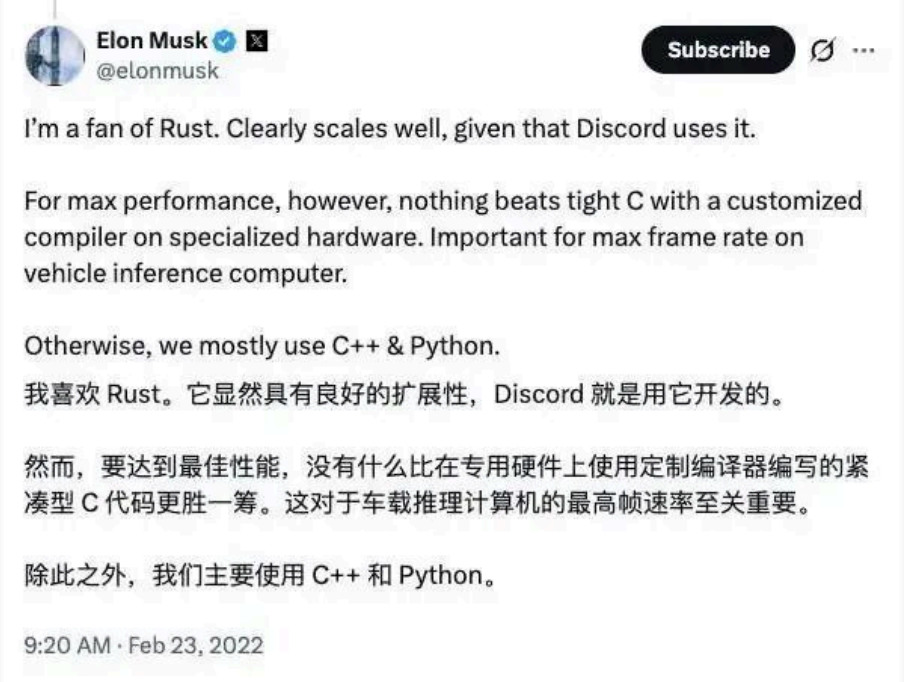
摘要：

Rust 移除垃圾回收、发明所有权模型，是“革命中的革命”。

“真男人自己管理内存。”

11 月 10 日，面对网友“为什么用 C/C++ 重写，之前用的是 Rust 吗”的追问，Elon Musk 用这句话回应了 xAI 的技术选择。

四年前，他还公开表示自己是 Rust 的拥趸，认为 Discord 的实践证明了它的扩展能力。不过，他当时也保留了一个判断：在专用硬件上追求极致性能，精炼的 C 代码配合定制编译器，依然无可匹敌。



如今，随着他宣布弃用 JAX/SGLang、用 C/C++ 从零重写强化学习与推理技术栈，这场关于性能、内存管理和语言选择的争论，又被推到了台前。



在 DTrace 联合作者、Oxide 联合创始人 Bryan Cantrill 看来，Rust 的价值，恰恰要放回过去几十年积累的工程负担中理解。许多语言为了让开发者更轻松，把复杂性藏进运行时：写代码时少操一点心，部署之后却要为垃圾回收、运行时错误和难以预测的性能付出代价。当这些软件成为下一层系统的地基，过去的便利就可能变成后来者无法绕开的技术债。Rust 受到追捧，一个重要原因就是它尝试扭转这笔交易：要求开发者提前想清楚所有权和内存关系，用开发阶段的约束，换取运行时的安全与可靠。

到了 AI 编程时代，这种约束反而更有价值。LLM 可以反复修改代码，编译器则不断指出问题，Rust 陡峭的学习曲线由此显露出另一面。不过 Cantrill 的警告也正在这里：Rust 可以拦住一部分错误，却拦不住 LLM 源源不断地制造复杂性。一个系统可以通过编译、符合内存安全要求，同时充满多余的模块、层层叠加的抽象，以及永远做不完的“改进”。

人会因为脑子装不下整个系统，设法让它更简单；LLM 却没有同样的压力。你让它精简，它可能再派一万个 Agent 来“优化”。Steve Yegge 的软件工厂 Gas Town 就陷入了这样的循环：原本要替人开发软件，最后却只顾着开发自己。按照 Yegge 的自述，Opus 4.7 总觉得“再做两件事就好”，不断修改 Gas Town 本身，始终无法进入真正干活的状态。工厂一直在扩建，却迟迟不肯开工；代码越堆越多，Token 越烧越贵，最终把自己拖垮了。

在 Cantrill 看来，这暴露了 AI 编程更深的风险：模型可以不停写、不停改，甚至把“简化系统”也变成一轮新的扩建。Rust 能检查内存使用是否安全，却无法替人判断，这座工厂究竟还缺一块砖，还是早就该停止施工。

本文基于该演讲视频整理，经 InfoQ 编辑。

每一分开发便利，都是运维者买单

先给大家一点背景，我在 Sun Microsystems——一家现在已经不存在的计算机公司干了 14 年，之后我去了一家云计算先驱公司 Joyent，它在 2016 年被三星收购。然后我们在 2019 年创办了 Oxide。这就是我整个职业生涯的跨度。

我入行那会儿，用事后诸葛亮的眼光看，几乎算是史前时代。那时候大家都觉得面向对象就是未来，但语言对它的支持糟糕透顶。那是 Java 出现之前，只有 C++。如果你觉得现在的 C++ 很糟，那你真该看看九十年代的 C++，更是烂到令人发指。

我很早就认为：编程语言这东西，给不了我太多东西。

我职业生涯是从 C 起步的，1996 年那会儿没人觉得 C 是未来，但我想写的操作系统内核软件，毫无疑问只有 C 可用。但软件世界的其他地方不是这么想的，诉求都集中在语言上：怎么让软件更容易写？怎么让更多人能写软件？

这些诉求听起来都很合理，我们确实希望软件更平民化：一门心思优化开发体验，降低了认知门槛，让开发者不必懂那么多。从汇编和 C 那个年代走出来，这听着很值得称道。可这些东西全都是一笔隐性交易：我们让开发者更轻松了，却让运维者难受得多。这个张力，从我九十年代写 C++ 的时候就埋下了，在今天的 LLM 时代又以一种全新的方式冒了出来。

垃圾回收（Garbage Colletction）就是绝佳例子。垃圾回收让开发者省事，可一旦真要运维这套软件，麻烦就来了，它会伴随软件的整个生命周期。我一个从显式内存分配年代长大的人，都觉得这事有点荒唐：我们居然要把“什么时候要内存、什么时候用完”这种负担从工程师身上卸下来。拜托，这又不是分裂原子。我想要点内存，用完了，还给你，这不难吧？可不行，太“难”了。

这对某些软件确实很棒，但对基础设施类软件，那些要承载其他软件、要扛负载的软件，就是跟你对着干。我认识的那些在垃圾回收语言里写数据库的人，首先，我为你感到难过，你花了大量时间去“讨好”垃圾回收器：有一块自认为没人引用的垃圾，拼命希望垃圾回收能发现它，不然垃圾回收就会 stop the world，给你带来延迟。更糟的是，你以为那是垃圾，其实不是，你手里还攥着引用。写过 JavaScript 的都懂，用闭包太容易创建出你看不见的对象引用了。然后你骂垃圾回收不停地跑，其实它根本没在回收垃圾，它现在是个堆扫描器，纯属南辕北辙。

动态类型也一样。不用预先定义类型、运行时再确定，对开发来说很爽。如果你从静态类型语言转到 JavaScript，那感觉就像进了裸体营地。你从小穿惯了衣服，突然大家都把衣服脱了，一开始是有点刺激，但人穿衣服是有原因的。动态类型也许让某些软件开发更容易，但现在你在生产环境遇到的问题，本该是编译错误，比如本来只是个手误把属性拼错了，现在却要在运行时爆出来，这就是为什么没人该用 Node.js 写基础设施软件。

还有 M:N 线程，我希望大家都不认识这东西。它号称“你能创建几百万个线程”，你真去创建几百万个，一切都卡死了，又是那种为了迁就开发者、却让系统更难理解的东西。

异常也是。在代码深处动态抛异常，感觉很好，不用担心错误处理了。可一旦真出了错，异常就像草丛里的蛇，很难正确处理。尤其 JavaScript 里，如果异常是因为属性拼错导致的类型错误，你根本不知道该怎么处理。

ORM（对象关系映射），“我们想让访问数据库超级简单”，对开发者很棒，开发速度飞快，甚至都不用知道自己是在跟数据库打交道。可真正在生产里处理这摊子的人心想：我就知道，他们不知道自己面对的是数据库。我这辈子花了太多时间在生产环境里调试别人的代码，总觉得这世界有一种因果失衡：一定有人制造的 bug 远多于他们调试的 bug。等你遇到这些人，很难开

口说“嘿，我读过你很多代码，顺便问一句，你知道那些东西全都是数据库查询吗？”这也情有可原，因为抽象就是不让你知道，好让你开发得更快，但代价是引入了所有这些复杂性。

RPC（远程过程调用），几乎从定义上就是个坏主意：你有个远程的函数调用，然后说“别担心你要走网络，就当调个函数”，这让你一不小心就造出一个病态系统。

抽象是软件的全部，而坏抽象会爆炸

这些都是抽象稍微出了问题的例子，所以让咱们聊聊抽象，因为抽象正是软件的全部。我们看不见它、闻不到它，却对什么是干净的抽象有很强的审美。一个好抽象能让我们造出非凡的东西，紧凑到人们甚至不知道自己被抽象掉了。这跟 ORM 那种“不知道”不是一回事：好抽象是无缝到根本看不见，不会在生产环境里引发涌现行为；坏抽象则会泄漏、会爆炸，产出的系统极其脆弱。

当你在开发阶段为开发者过度优化，想把开发者从现实和后果中屏蔽出去时，几乎不可避免地会造出一个泄漏的抽象，而收拾它的人是运维。

垃圾回收就是最好的例子。它本来不该泄漏，前提是我们真的永远不用操心它。但每个 Java 性能问题最后都是“你得换个垃圾回收器”，或者“换个算法”。我们简直是在搞降神会，从替代垃圾回收器的超自然世界里召唤灵魂，这就是恐怖地泄漏的抽象。它对很多系统没问题，但当我们试图构建基础设施，那种我们在其上构建更多软件的软件，它就会坏得很惨。

坏了，它们就引入复杂性。复杂性和抽象是软件工程的一对偶极子。好的抽象把我们蒸馏到问题剩下的本质复杂性上，坏的抽象诱发偶然复杂性。真正的问题在于，复杂性不只是累积，它还会爆炸：坏抽象诱发的偶然复杂性会变成下一层必须处理的本质复杂性。你的坏抽象创造了一个糟糕宇宙的新规则，而我们发现自己就困在里面。

举个非常具体的例子：基本输入输出系统（BIOS），它是系统里非常底层的固件，后来演变成 UEFI。x86 或者 ARM 系统启动时，就会执行这段固件。这挺让人失望，因为 ARM 本来根本不需要模拟这套行为。如果你在实现一项技术，想取代一个你觉得比你走得更远的技术，千万不要假设围绕它运转的复杂性就是本质复杂性。ARM 那帮人觉得，我们 ARM 也必须有 UEFI BIOS，这玩意儿是本质复杂性。才不是，它是偶然复杂性，你其实根本不需要它。但当每个人都用的时候，你就会觉得它不可协商。我们日常要处理的大量复杂性都像是不容置疑的本质复杂性，但其中很多其实是过去选择造成的偶然复杂性。

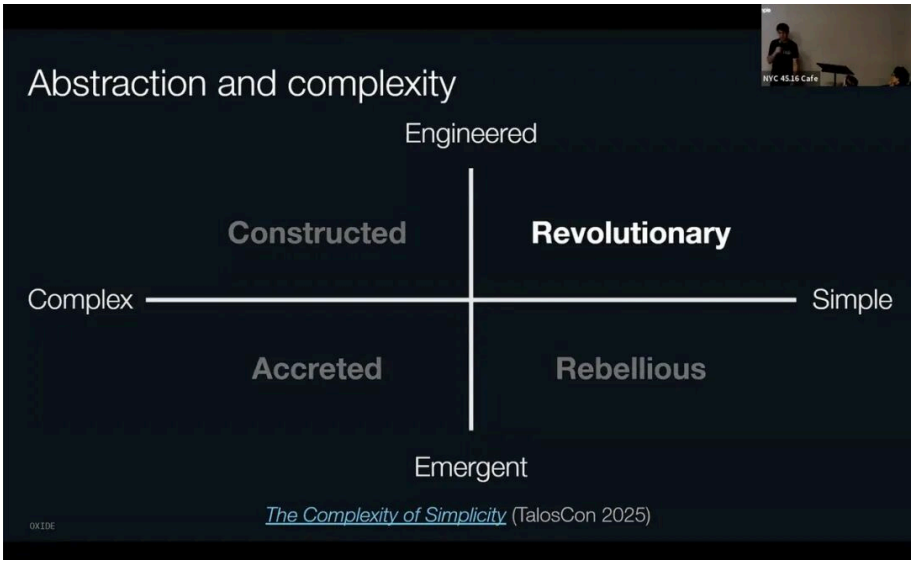
大概一年前我把这种分类做成了一场演讲。我画了两个轴：一个从复杂到简单，从“搞得一团糟”到“优雅”；另一个从涌现到工程化，衡量我们对决策有多刻意、多深思熟虑，而不是多被动、多应激。这样就得到四种系统。

左上角是工程化的、复杂的系统，我们叫它“构造型（Constructed）”。Fred Brooks 那本 60 年代写的《人月神话》到今天仍然切中要害，它描述的就是一个超大型构造型系统的风险：60 年代 IBM 那个 360 系统，规模巨大，很多人同时在上面工作。

它们会变得非常复杂，然后激发出“叛逆型（Rebellious）”系统（译者注：位于第四象限）：我们真的需要做所有这些事吗？要不全扔了？这些复杂性解决的是我们根本没有的问题，“来加入我的叛乱吧”，这其实就是私募股权那种“通过摧毁来创造价值”的幻想。大多数时候他们只是毁灭，但当这类系统真的奏效时，确实丢弃了一堆你并不需要的东西。Multics 到 Unix 就是典型。Unix 这个名字本身就是对 Multics 被阉割的双关，一个刻意叛逆的系统：我们要靠做得少得多来创造点什么。叛逆型系统会在“删得太多”的时候失手：它说“这个其实承重”，而那个被删掉的东西恰恰让系统无法交付了，好在大多数时候被丢弃的都是偶然复杂性。

然后系统常常会落到第三个象限：“累积型（Accreted）”系统。它是涌现出来的，不是设计出来的，但同样复杂。很多系统一开始是构造型或叛逆型，最后都变成累积型。UEFI BIOS 就是百分之百的累积型系统：高度涌现、非常复杂，你极难把本质复杂性从偶然复杂性里择出来，这就是技术债。

有些系统一出生就是累积型的，连推广它的人都说“这很遗憾，但它确实解决了问题”。累积型系统作为地基非常糟糕：里面太多偶然复杂性搅在本质复杂性里，而这份偶然复杂性会变成架在它之上的东西的本质复杂性。不过，被累积型系统压得够久、够重之后，灵感可能会降临：我要做点彻底激进的事，靠创造来创造价值，而不是靠摧毁。



你会拉上一小撮人，给他们专注的时间，去做一件有点野的事。我们要改变抽象，而不只是减少抽象。几乎所有编程语言都落在这个范畴里，因为发明一门语言，几乎就是在发明一种新的抽象。否则你干嘛不直接用 Lua 嵌进去？如果你被系统的累积压得喘不过气，可以去这些“革命性（Revolutionary）”系统里找灵感。

当年的革命，怎么成了今天的技术债？

但这里有个悖论：你正在对付的那个累积型系统，很可能不久前自己就是个革命性系统。它们起步时是革命性的，最后却变成了当初想要颠覆的那个东西。

Java 就是个绝佳例子。去翻翻 1995 年 Gosling 那份白皮书，那时它还叫 Oak，你能体会到 Java 当年那场真正的革命。Java 是对 C++ 的革命，而 C++ 是一团乱麻。C++ 是“出生即累积”的典型：当创造者觉得自己得为这个创造向全世界道歉时，你就知道它是累积型出生的。

Java 是一场革命，但没花太久就变了味。

如果你现在整天泡在 J2EE 系统里，绝不会说“这些抽象多优雅”，只会说这就是一团彻底的乱麻。2005 年的 Ruby on Rails 就是对 Java 复杂性的反作用。Python 也如此：它最初是作为教学语言设计的，有很多好东西，也有太多复杂性。今天你不会把 Python 称作革命性系统，尽管它起步时是。

为什么这些系统最终都走向累积？因为累积成了把这些东西当作生产基础设施来运行的必然属性。二十年前那可能是个好主意，但现在我正试着拿它当地基往上盖东西。

Java 优化了垃圾回收，它是为开发阶段的开发者优化的，我祝 Cassandra 的开发者永远不用操心内存。可我现在要在生产环境里对付 Cassandra，做垃圾回收的降神会，想压低延迟，一边

还盘算着往右上角那片革命性系统的象限看。

我不反对垃圾回收本身，我反对的是，把它当作基础设施部署到生产环境之前，对它会变成什么样一无所知。结果就是，很多革命性系统起步时是革命性的，却实际上变成了一场奔向累积的加速跑。

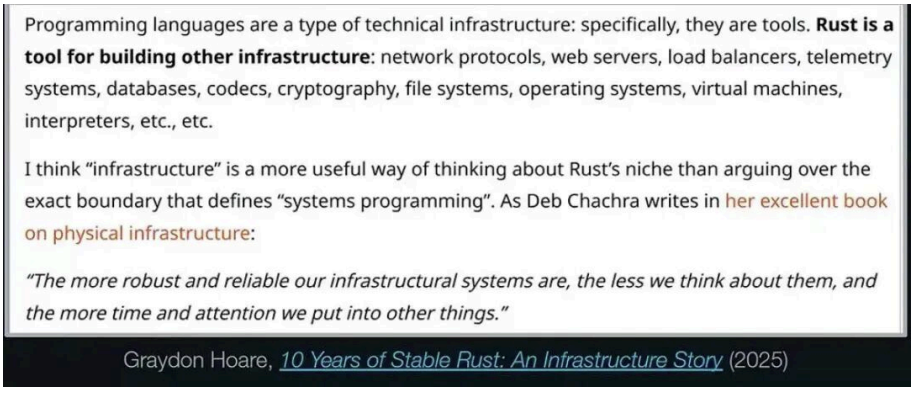
幸运的是，今天你在这里，是因为有一场更深的革命正在酝酿。对任何系统，回到原始文档总是有帮助的。创造者当初到底想做什么？你能学到这个系统为自己做决策时用的那套标准，这非常重要。

就 JavaScript 而言，你根本找不到 Brendan Eich 关于 JavaScript 的基础性文档，这本身就说明了一些问题。JavaScript 某种程度上是“出生即累积”的，他们连名字都没保住，本来叫 LiveScript，后来改成了 JavaScript。如果你曾经是个编程语言新手，问“抱歉我没跟上，Java 和 JavaScript 有关系吗？”没有，它们连表亲都算不上，它们是营销上的表亲。所以回到原始文档很重要，它能告诉你决策为什么这么做。

Rust 真的非常迷人，原因有很多很多。

Rust 的发明者 Graydon Hoare，我和他的渊源说来有点奇怪。我在 Sun 做过一个叫 DTrace 的东西，能在生产环境里观察系统，后来被几个开源项目“致敬”了，其中一个叫 SystemTap。那个项目里只有一个人让我觉得“这人真不错”，而当我发现他离开项目时，我内心阴暗地感到一阵狂喜，因为那意味着我不用再跟那个人打交道了。那个人就是 Graydon，他离开是因为想去做一门新的编程语言。

那是 2004 到 2006 年间的事。我当时想，这太荒谬了。新编程语言已经被过度优化成“让开发者爽”，而对我这种需要造出生产环境可依赖的基础设施的人来说，它们什么也没给我。我那时在用 C，C99 就是我能等到的最大变化，我几乎认命了。



> 我认为，与其争论“系统编程”的确切边界，通过“基础设施”来理解 Rust 的定位更有帮助。正如 Deb Chachra 在她那本关于实体基础设施的优秀著作中所写的：“我们的基础设施系统越稳健、越可靠，我们就越少需要考虑它们，也就越能把时间和注意力投入其他事情。”
>
> ——Graydon Hoare，《Rust 稳定版十周年：一个基础设施的故事》（2025）

但幸运的是，Graydon 真的去把 Rust 造了出来。他后来回顾 Rust 稳定十周年时写了一段话：“编程语言是一种技术基础设施，具体来说，它们是工具。Rust 是构建其他基础设施的工具：网络协议、Web 服务器、负载均衡器、遥测系统、数据库、编解码器、密码学、文件系统、操作系统、虚拟机、解释器等等。”我是个系统人，看到“等等”就想，还有别的吗？这不就是全部软件了吗？

用“基础设施”来理解 Rust 的生态位，比争论“系统编程”的精确边界有用得多。就像 Deb Chachra 那本讲物理基础设施的书里写的：我们的基础设施越鲁棒可靠，我们就越不会去想它，从而把时间精力放到别处。你不需要惦记的基础设施，才是真正好用的抽象。

不会读 Rust，还嫌 Rust 代码丑?

这一点很关键，因为 Rust 做过一些非常了不起的事。如果看那些“为开发者服务”的特性，Rust 至少有两个：绿色线程和垃圾回收。当年我得知这两点时，感觉就像收到了来自那座偏远岛屿的消息，而那座岛正在烧成灰。我当时面无表情地想，好吧，当然。但接着疯狂的事发生了：Rust 把绿色线程删了，把垃圾回收也删了。

移除垃圾回收在我看来是一个极其有趣的时刻，因为它实际上就是所有权模型的发明。我知道这不是 Rust 独有的，我一说独有，就会有人跳出来说 Simula 在 1967 年就干过、Clean 也干过，好吧，我向所有 Simula 和 Clean 程序员道歉。但它绝对不在主流里，这几乎是“革命中的革命”。

Rust 是 Graydon 在 Mozilla 开发出来的革命性系统，但它也开始积累偶然复杂度。Graydon 的目标是造一个作为基础设施工具、能导向“移除”的软件。虽然移除不是他亲手做的，但他创造了一种文化，这种文化导致了垃圾回收和绿色线程的移除，并在很大程度上挡住了异常机制。Rust 把“基础设施”当作约束，先说我们要让产物足够鲁棒、能当基石，然后在这个界限内尽量把开发者体验做好。我们不会为了让开发者舒服，就交出一个不鲁棒、不安全、不可靠的产物。

Rust 社区历史上的很多紧张关系，就来自“这条线到底画在哪”。总体上说，Rust 做得极其出色。我不是说写 Rust 不难，我是说那不是优先级。我们实际上是把认知负担移回给开发阶段的开发者，你得真的去想你的内存。这里有一笔大交易：如果你按“谁拥有什么”来组织程序，Rust 就给你安全。我之前几次顺嘴黑垃圾回收时说“人们只是不会 malloc 和 free”，那有点轻佻。实情更复杂：你在管理内存时，是在一门目的论上不安全的语言里工作，不只是 malloc/free，还有内存损坏、缓冲区溢出。Rust 说的是：我要强迫你，程序员，指明谁在什么时候拥有什么，作为回报，我给你安全。

我 2018 年才接触 Rust，正好在 non-lexical lifetimes 之前。那时编译器天天把你按在地上摩擦，它建议你“把这个改成 mut”，你改完它又说：“谁让你这么干的？”是你自己啊，我只想让代码编译通过而已。我犯了很多 C 程序员都犯的错：我没有真正理解那笔交易，没有去想“我得换一种方式组织程序”，而是把我想当然的结构硬塞进 Rust 的所有权模型里。你确实能在 Rust 里写双向链表，但你真的不该写。双向链表从定义上就是“两个东西拥有同一个对象”。有个很棒的博客系列叫《So You Really Want to Write a Doubly Linked List in Rust》，通篇都在劝你别干，但“你非要的话，来吧”。对 C 程序员来说，这相当于一口气抽完一整包烟。

最终我被击败了。我意识到，算了，我不写双向链表了，我按你的方式来，我眼泪汪汪地只想让它编译通过。我拿一个项目试 Rust，本来想先让它跑起来别管性能，结果它跑起来后，比我写的 C 还快。我几乎被自己冒犯了：我花了很多时间写的 C，怎么会比 Rust 慢?

我去问 Steve Klabnik，这里面肯定有什么精妙的编译器技巧吧。我列了一堆猜测，他说都是好主意，但我们一个都没做。真相简单得多：是抽象的力量。

你想在 Rust 里用 B-tree，用 BTreeMap 就行。我们对此太习以为常，以至于忘了这件事有多么了不起。在 C 或 C++ 里用 B-tree，就像带着枪进幼儿园。因为 B-tree 依赖在内存里移动数据，而 C 里没有“谁拥有什么”的检查。你去找找 C 的开源 B-tree 库，很难找到，因为契约太复杂了。但在 Rust 里，正是因为有所有权模型替我们检查这些问题，我们才能放心地使用

BTreeMap。这有点反直觉：一方面它对你要求更多，另一方面它给你的远多于拿走的。这才是 Rust 真正具有革命性的地方。

尤其在 LLM 时代之前接触 Rust 的人，会有那种非凡的感觉：我的 Rust 一编译通过，它就能工作。重构也一样。在很多语言里重构是自担风险，我觉得重构常常得不偿失，因为它引入风险。但在 Rust 里，你重构完只要编译通过，它大概率就能工作，编译通过本身就是一个数据点。

当然，这也意味着学习曲线陡峭。你刚学 Rust 时，人们说“我害怕那个可怕的编译器”，结果你发现它超级友好，终端里有颜色、有 ASCII 艺术精确指出你的错误。这个友好的编译器是在积攒社交资本，准备在你最黑暗的时刻一次性提现到接近零。学习曲线是陡的，但它给了你巨大的回报。

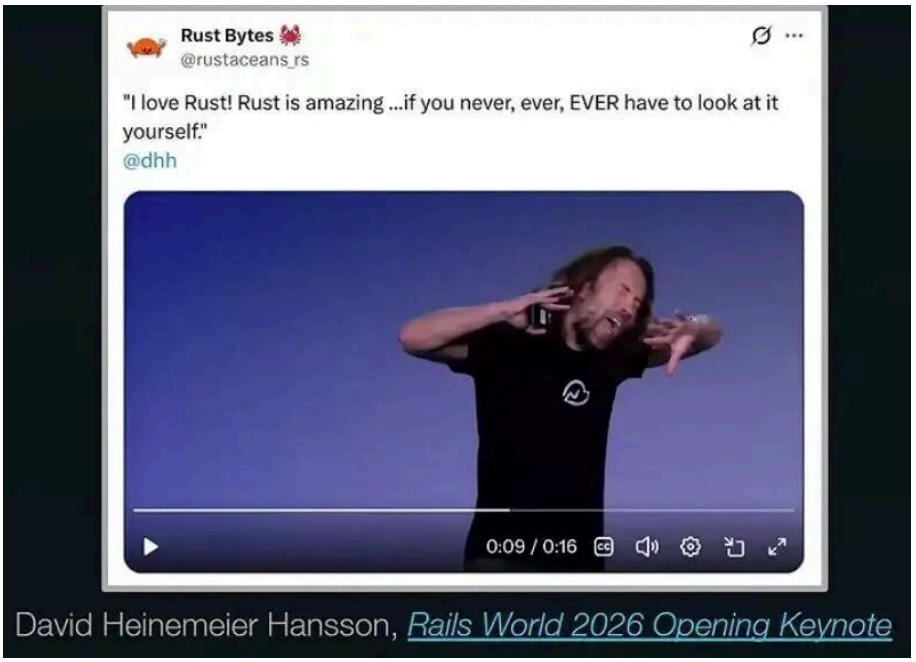
说一个反直觉的结论：Rust 那个被诟病的“学习曲线”，在 LLM 时代反而成了优势。认知负担从生产端转移到开发者这一侧，这其实是件大好事。代码只要编译通过，我们就能对它的正确性说点什么。这种反馈对 LLM 来说简直是完美的。

还有一点：Rust 关于构建鲁棒、安全、快速软件的那些价值观，作为一种基础设施工具，它吸引的是同类的心智，这是 Rust 最重要的事情之一。你走进 Cargo 生态，那里有极其出色的软件。我敢说你们都有过自己的 Serde 时刻，就是那种“我的天”的感觉：一行代码解析 JSON，而且比我以往任何时候解析 JSON 都更鲁棒。我们有很多这样的 crate，正因为我们吸引了那些同类的心智。这个生态给了 LLM 一个非常鲁棒的基础，让它可以放心去 YOLO。你坐在鲁棒的地基上，vibe coding 起来容易得多，而 Rust 的地基极其鲁棒，同时产物本身更好、更快、更安全。

当然，你会想，vibe code 出来的东西有顾虑？没事，它是 Rust 写的，这帮我解决了。不，并没有解决，但它确实很有吸引力。而且我认为很多人会说，“太”有吸引力了，因为接下来就有人登场了。

David Heinemeier Hansson，Rails 的缔造者 DHH，他现在相信一切都应该永远 vibe coding，在 Rails World 上做了一场关于自己怎么用 Rust 的主题演讲。你第一反应是：太好了，DHH 在用 Rust，我们为什么不张开双臂欢迎他？但你最好先把那场演讲看完再说。

实际上，他一边用 Rust 做所有事情，一边在同一时刻贬低 Rust，而且贬低的是 Rust 的美学。美感这东西见仁见智，但有些“见仁”的人，事实证明就是个蠢货。DHH 觉得代码很丑。我特别“欣赏”这一点：这个人读不懂 Rust，还以读不懂 Rust 为荣，只靠 vibe code，然后反过来抱怨代码的质量。你想欣赏一门语言的诗，你总得会说这门语言吧。



他说“我用 Rust 是因为性能惊人”。对，但为什么性能惊人？性能惊人是因为你用的是 BTreeMap 而不是 AVL 树。为什么你能有这个东西？因为所有权模型。你之所以能用 Rust 编程，是因为很多人多年来在这门语言、环境和生态里付出了极其艰苦的努力。你从中受益，同时又在贬低它，这是相当冒犯的。如果你是 Rust 社区的新人，请不要这么做。

AI 越勤快，系统越复杂

这让我想到 LLM 的一堆问题里的一个：LLM 缺少人类最重要的特征之一，它们不懒。Larry Wall 说懒惰是程序员的三大美德之一，另外两个是傲慢（hubris）和鲁莽（impertinence）。我们给操作系统起名叫 Hubris，也许确实是因为我们自己的傲慢，但也确实是在致敬 Larry Wall 说的那个美德。

Larry Wall 是 Perl 的作者。他说的懒惰指的是：把系统装进我们脑子里、受限于自身认知边界的需求，会驱使我们走向优雅而简单的抽象。我们当年在一个不断堆积的系统里发起革命，部分原因就是我们理解不了那么多复杂性。在一个堆积出来的系统里是没有第一性原理的，而第一性原理往往就来自懒惰。我敢说在座不止我一个人对三角学只记得一件事：$\cos^2 x + \sin^2 x = 1$，剩下的考试时现推。只要掌握第一性原理，三角函数照样能考得不错。

而 LLM 完全没有这个。LLM 热爱工作，所以它们强烈偏向复杂，不只是复杂问题，而是复杂本身，它们会乐呵呵地为你制造复杂性。这很糟糕，我想让它懒一点。当然你可以像对付经典 LLM 那样抽它一巴掌，让它“懒一点”，它会道歉，然后非常努力地去变懒。

我的意思是，去思考 Rich Hickey 那套著名的“吊床驱动开发”（hammock driven development），真的去琢磨这个系统。结果它说：明白了，我派一万个 agent 去琢磨这个系统。算了，可能还是别了。

更糟的是我们还有所谓的管理咨询顾问效应。我上学那会儿，管理咨询是数量可怜的一小撮院校本科生的常见去向。你要是 90 年代上了常春藤，有朋友去当管理咨询顾问。你可能会像我一样想：你二十分钟前还在玩酒桶倒立（keg stand），凭什么有人听你的？但这个理念，麦肯锡就是极致代表，是“我们要找最优秀最聪明的人”。对，就是 McNamara 那个贬义上的“最优秀最聪明”：最优秀最聪明的人，却不懂越南其实是一场内战。

他们去给 CEO 和高管层做咨询，最后往往形成一种有毒关系。要么他们自己根本不知道自己在干嘛，一个 22 岁、聪明但什么都不懂的年轻人居然能替你运营业务？要么更常见的是：我其实

就想让你背书我本来就打算做的决定，或者“帮我打开眼界，我们可以把阿片类药物更好地营销给医生，太感谢了麦肯锡”。

你手里那些 LLM，就是你的小麦肯锡咨询顾问。你要是放任它们，它们会和你形成一种共生依赖。它们很难把事情“做完”，就像管理咨询顾问希望周一出现在你面前。麦肯锡出了名地周一到周四驻场，他们想周一再回来。你问“做完了怎么办”，他们说“什么叫做完？永远有更多事可做”。这是实实在在的危险，因为 LLM 明里暗里就是为参与度（engagement）设计的。

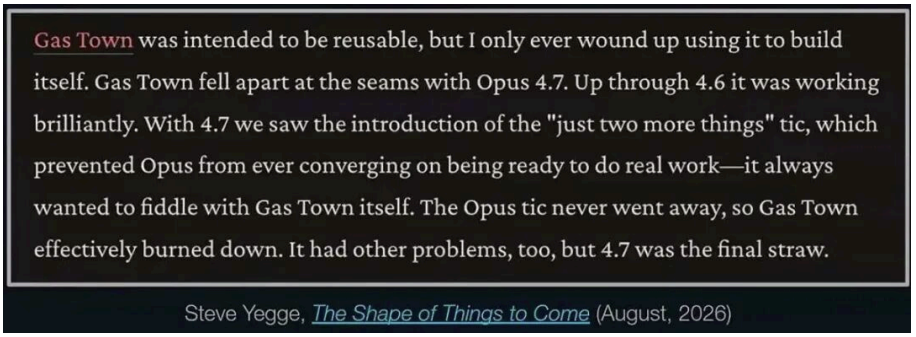
管理咨询顾问还是个厚道的比喻，它也可能是一台意外造出来的老虎机。人们常说“我用 LLM 用得精疲力尽”，那你有没有在赌场待过九个小时？是不是同一种感觉？这些东西为参与度而设计，这本身不算错，我们其实也想要这个。但当一个为参与度设计、长得像管理咨询顾问、又没有人类“懒惰”特质的东西来替你设计软件时，你最好小心点。

因为你可能得到能跑的东西，然后你发现问题，它说“别担心，周一还有 200 个咨询顾问落地，我们全给你重写”。你说这不是我想要的，我想让这东西更紧凑更优雅。“明白，我们派 300 个过去，让它更优雅”。所以你得意识到它可能是台老虎机，安息吧，Gas Town。

为什么 Gas Town 最终会走向失败

Gas Town 是 Yegge 开发的一个软件工厂，风格大概像是理查德·斯卡里遇上《疯狂的麦克斯》。它是用 Go 写的，不是 Rust，但 Yegge 吹嘘过一件事：“我从没看过这些代码，也不打算看。”你听了就想，真的吗？你就让那台麦肯锡老虎机把代码全写了？行吧，那后来呢？

后来就悄悄发生了，Gas Town 烧没了。我觉得这事挺有意思的。第一，它不知道该怎么停下来。你看过 LLM 互相打电话、却不知道怎么挂断吗？就是这个，只不过发生在你的软件上。它永远不收敛，不停给自己找新活儿干。与此同时，旗子倒下、计价器还在跑，这也很贵，而且 Opus 那档模型的消耗始终没停。



> Gas Town 原本被设计成一个可复用的工具，但到头来，我只用它来构建它自己。到了 Opus 4.7，Gas Town 开始全面崩溃。在 4.6 及此前的版本中，它一直运行得非常出色。可到了 4.7，模型开始出现“再做两件事就好”的毛病，导致 Opus 始终无法收敛到可以开始干正事的状态——它总想继续折腾 Gas Town 本身。Opus 的这个毛病始终没有消失，所以 Gas Town 实际上就这么烧毁了。它也存在其他问题，但 4.7 是压垮它的最后一根稻草。
>
> ——Steve Yegge，《未来事物的形状》（2026 年 8 月）

LLM 无可争议地极其强大。我知道确实有人说“我永远不用”，但我们太沉迷于乌托邦或反乌托邦的未来想象，反而忘了：这些东西在理解力上是爆表的。如果你手头有一个代码库，我建议用 LLM 去问关于你自己软件怎么工作的问题，而且这些答案可以随手验证。你不用去担心老虎机效应、麦肯锡效应那一套，就问“这部分是怎么运作的”，它给出的答案出奇地好，还带出处。所以它在理解层面极其强大，生成层面也强大，但你得睁着眼睛用。

我们同事 Rain 有个技巧我特别喜欢：她不让 LLM 替她写注释，注释要自己写。理由是，如果没法为它写出一条注释，说明她根本没搞懂发生了什么。她可以跟 LLM 讨论，搞清楚状况，再自己把注释写出来，用这个动作逼自己真正去看它做的每一件事。

我觉得这是个很有意思的技巧，我们还会发现很多这类“如何真正有效地使用这些工具”的门道。但核心是：你要把它当工具用。任务定义得越清晰，你成功的概率越高；你把口子开得越大、让它钻进复杂地带，它就越疯，Gas Town 被烧掉的概率就越高。

而且我们 Oxide 坚信一件事：LLM 可以让我们变得更严谨。程序证明这事，如果当年你告诉我，编程语言开发会给我这个系统程序员带来什么，我做梦都想不到程序证明会离我这么近。形式化验证？绝对不可能。结果呢，形式化验证现在真的成了实打实的东西，部分原因就是 LLM 能扛下大量重活。但我们不希望 LLM 取代软件工程的核心本质，那个本质就是创造抽象。

学 Rust，代码仍然重要

所以我要说，如果你是 Rust 新手，去学 Rust。哪怕你用 LLM 生成了大量 Rust 代码，你也不希望自己患上习得性无助。你不希望成为那种没法调试自己代码的人，不希望成为那种必须靠 LLM 才能看懂 LLM 写了什么的人。

你应该学 Rust，而且我要告诉你：你能学会 Rust。任何学 Rust 的人都要先跨过一道坎，那种“Rust 高不可攀”的神秘感。但如果你心里想“我不知道我能不能学会 Rust”，你能学会。好消息是，现在有个很棒的新工具能帮你学东西，它叫 LLM。

我们正处在全新的领域。我有三个孩子，看着 19 岁那个上大二的儿子天天用 LLM 学习，特别有意思。他告诉我：我有了一个家教，有了一份学习指南。当然，因为高等教育现在同时经历八场不同的存在主义危机，他所有的考试都改成了课堂手写，还放在法拉第笼里，你没法作弊。但对他来说，把 LLM 当学习辅助工具，效果是惊人的。用 LLM 来学习，多么激进的想法。但它是可行的，完全可以用这些东西来增强我们的理解。

所以我真心劝所有人：请去学 Rust。不要以“我不读 Rust”为荣，如果你真以此为荣，拜托别在 keynote 上讲。如果你非要在 keynote 上讲，也别对着一群用别的语言的人讲，那看起来既刻薄又奇怪。

还有，记住 Rust 的使命，记住 Graydon 的使命。你造的东西，是别的软件要依赖的东西。我相信 Rust 是一场如此重要的革命，我认为我们正处在软件领域的某种文艺复兴。我知道现在感觉不像，感觉我们正在被 LLM 去人性化，但其实没有。因为我们正在生成后代将要依赖的基础设施，代码依然至关重要。

Q & A

观众：你在 Oxide 或者个人使用中，把 LLM 看成是做减法还是做加法？

Bryan：肯定是加法，但它需要纪律才能成为加法。这有点像那句据说是 Henry Ford 说的话：你觉得你能行或者你觉得你不行，两种情况下你大概率都是对的。如果你认为 LLM 是加法，你就会用出加法的效果；如果你认为它是去人性化的，你大概率也会用出去人性化的效果。

LLM 在很多维度上都极其非凡，而我们其实还没真正理解它。我还记得互联网出现之前的日子，在维基百科之前，你只能靠酒吧里的争论。而现在你可以随时问任何问题，得到一个来源可靠的答案，这太惊人了。

我写了 RFD 576，讲的就是 Oxide 内部怎么用 LLM，我特别建议大家去读。让我惊讶且沮丧的是，真正把这方面的思考成文、制度化的公司少得可怜，或者写了但没公开。

有一件事我特别推荐其他公司也做：Oxide 所有公开写作，我们都坚持要“Pangram clean”（Pangram 是一个 LLM 检测器），必须是人写的。原因很简单：别让你的公司或你自己发布 LLM 写的东西，因为大家都看得出来。不只是那些破折号，而是信息密度太低了，你的大脑会直接拉弹射把手，我真的读不下去那个句子。你这样做，是在阻止自己的信息传达出去。别让你的公司这么干，别让你的基金会这么干，说的就是你，Rust Foundation。他们有一篇博客是 LLM 写的，我们点出来了，他们改了，这很好。至于我们怎么用 LLM，我强烈建议每家公司开发自己的评分标准（rubric），这个结构可以拿去用，不是专有的。

观众：你和 Rich Hickey 似乎在很多事上看法一致，比如支持简单性、反对 ORM。但他开发 Lisp，那是一门动态语言，还带 JVM 上的垃圾回收。你怎么解释你们俩在垃圾回收和动态类型上的分歧？

Bryan：可能只是我们写的软件类型不同。我只通过演讲了解他，他的演讲非常精彩，但我不总是同意他说的每一句话，因为我们对系统的视角不一样，我写的软件在栈里更深的位置。他总不会用 Lisp 写操作系统或固件吧？

另外我想说，我在垃圾回收这个问题上可能有点说过头了。我想向那些被我不小心冒犯到的、支持垃圾回收的人道个歉，垃圾回收有它存在的权利，只是它不适合我想写的那类软件。

观众：我们并不是在写持久化 map、原子 ref 和全套 Clojure，那样的开发者很少，但理念似乎是相通的。最近有篇新博客叫《Simple Made Inevitable》，它基本上是在说，有了 LLM，Rust 获得了一些好处，因为那道门槛被降低了，对 Clojure 也是一样。我想听听你对这个的看法，还有你觉得其他编程语言会不会也出现类似变化。

Bryan：我确实认为，这就是我们身处的有趣时代。有些系统一直被贬低，因为它们小，或者像 Clojure 那样，又小又有点怪。但 LLM 让人惊叹的一点是，一个社区的规模，已经不再是它软件发展的障碍了。如果开始出现一些系统开发，是带着“如何正确地与 LLM 交互”来设计的，我一点都不会惊讶。

但我也觉得这很危险，眼下这件事直接变成一团乱麻、把一切拖垮的概率相当高。我们正处在一个你以后会反复回味、甚至可能在法庭证词里回忆的时期。未来会有年轻人问你“那是什么感觉”，你会说：天啊，太诡异了。所以我不会因为一个社区小就贬低它，因为你现在有能力去做远超你规模的事情，只要你驾驭得当。

观众：那你怎么看现在软件工程里的“手艺”？我觉得在一家公司里，很难有理由花大量时间去所谓“传统编码”。我觉得享受写代码这个动作本身、并且保住自己的技能，是很重要的。

Bryan：百分之百同意。我会去别的机械化革命里找安慰，你看木工行业里电动工具是怎么被采用、又是怎么被拒绝的。我们不想丢掉自己的手艺，手艺非常重要。所以暗示“代码没那么重要”、暗示“我们不该盯着代码看”，其实就是在贬低这门手艺，这是个真问题。

我们的招聘流程非常“重写作”，我们会问候选人“你什么时候最不开心”之类的问题。我也知道现在有很多公司和管理极其糟糕的组织，你觉得自己“没法花时间把事情做对”，这真的很不幸，你值得拥有一个和你抱有同样信念的组织。我相信会再来一次文艺复兴，手艺会被重新表达、被重新背书，只是当下确实是个艰难的时期。

观众：你觉得 Rust 里还有哪些特性算是“错失的特性”，或者说被“回退”了？我个人会提名 async。

Bryan：我并不是很喜欢 async，原因有很多。async 取消安全性，集中体现了我在抽象层面反感 async 的那些问题。那些论证“我们不必担心 async 取消安全性”的说法，和 C 程序员说服自己“我们不必担心内存安全”的论证，惊人地相似。它基本上是无法推理的。

我们还遇到过另一个问题，我们称之为 futurelock，它同样是一种涌现复杂性。不过我想加一个重要的星号：我们遇到的问题实在太少了，以至于会专门去谈它们。在我们开发过的其他大型分布式系统里，失败简直是环境性的、无处不在的。而在 Rust 里我们完全没有这些。所以 async 取消安全性这些问题确实存在，但整体体验真的非常出色。

互联网根本没有这些反乌托邦的东西。回头看，一堆东西当时就觉得是个好主意，比如社交网络，感觉棒极了，有什么问题呢？然后就是“不不不，把它塞回盒子里去”。

我真的认为，等尘埃落定，LLM 很可能是一个“互联网级”的事件。每个时代的人都觉得自己独一无二，我们从农业社会过渡到工业社会，大概只用了一代半人的时间；美国电气化全国用了大约 15 年。收音机、电视，都带来了一堆问题，所以这个想法本身并不新鲜。

但油门踏板现在确实像是踩到了底板上。虽然我职业生涯里也有过其他类似时刻，但此刻座舱里每一个 dot-com 时代的警示灯都在闪。同时我也得说，我对前方的能见度，比职业生涯中任何时候都低。

观众：你是怎么在你的构建系统和流程里培养 LLM 纪律的？

Bryan：这个问题我会直接把你引向 RFD 576。它给出了一个非常好的评分标准，说明了我们正在用的东西。

来源：InfoQ

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次 VIP会员·7天畅读卡 | 大师课·入门串讲 扫码 免费领取

限免

111 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

1 条评论



比宇宙更遥远的那棵树

4小时前　中国四川省

太能写了，这么长

0　回复